

آموزش صفر تا صد - نسخه ۲۰۲۶

JavaScript + TypeScript

از صفر مطلق تا حرفه‌ای

از اولین `console.log` تا `typing` حرفه‌ای در پروژه واقعی

TypeScript Strict

Events و DOM

Async کامل

متغیر تا Closure

zod + React

Generics

۱۳ فصل در دو بخش · تمرین با جواب · چیت‌شیت دوتایی · گلاساری

فهرست مطالب

نقشه دوره دو بخشی	شروع دوره	1
Node، متغیر، انواع، عملگرها	فصل ۱ – اجرا، متغیرها، انواع	2
if/switch/for/while + توابع	فصل ۲ – شرط، حلقه، تابع	3
scope، hoisting، closure، arrow	فصل ۳ – Closure و Scope	4
destructuring، spread، map/filter/reduce	فصل ۴ – آبجکت و آرایه ★	5
Date، JSON، رشته، Intl فارسی،	فصل ۵ – رشته، عدد، JSON	6
TODO، پروژۀ، رویداد، querySelector	فصل ۶ – DOM و رویدادها	7
Promise، async/await، event loop	فصل ۷ – Async کامل ★	8
import/export، خطا، دیباگ	فصل ۸ – مازول و خطا	9
TS، annotate، unknown چرا	فصل ۹ – TS از صفر	10
interface، union، generics	فصل ۱۰ – Generics و interface	11
Utility Types، guards، satisfies	فصل ۱۱ – TS پیشرفته	12
React، zod، تایپ‌دار، fetch	فصل ۱۲ – TS در پروژه واقعی	13
چیت‌شیت دوتایی + گلاساری	فصل ۱۳ – چیت‌شیت و گلاساری	14

دوره جامع JavaScript و TypeScript – صفر تا صد 🎓

مخاطب: از اولین خط کد عمرت تا سطحی که در هر پروژه‌ای (Node، React، فرانت، بک) راحت JS/TS بنویسی
ساختار: دو بخش – JavaScript عمیق (فصل ۱-۸) و TypeScript از صفر تا عملی (فصل ۹-۱۲) – فصل آخر
چیت‌شیت و نقشه راه

🎯 چرا JS + TS با هم؟

- **JavaScript** زبان وب است: هر مرورگر، هر فریم‌ورک فرانت (React/Next)، و سمت سرور (Node). بدون آن هیچ فرانت‌کاری نیست.
- **TypeScript = JavaScript** + سیستم تایپ. امروز در آگهی‌های شغلی جدی، TS دیگر «امتیاز» نیست – **پیش‌فرض** است.
- ترتیب درست یادگیری: اول JS را واقعی بفهم (نه سطحی)، بعد TS – چون TS فقط «قوانین سخت‌گیرانه‌تر» روی همان JS است.

📖 نقشه دوره

بخش اول – JavaScript (فصل ۱-۸)

فصل	موضوع
فصل ۱	نصب و اجرا، متغیرها، انواع داده، عملگرها
فصل ۲	شرط‌ها، حلقه‌ها، توابع پایه
فصل ۳	توابع عمیق: scope، closure، callback، arrow
فصل ۴	آبجکت‌ها، آرایه‌ها و متدهای قدرتمند
فصل ۵	رشته‌ها، اعداد، Date، JSON
فصل ۶	DOM و رویدادها (فرانت واقعی)
فصل ۷	Async کامل: callback → Promise → async/await
فصل ۸	ماژول‌ها، مدیریت خطا و دیباگ

بخش دوم – TypeScript (فصل ۹-۱۲)

فصل	موضوع
فصل ۹	چرا TS، نصب، annotate پایه
فصل ۱۰	interface/type، union، narrowing، generics
فصل ۱۱	Utility Types، type guards، strict mode
فصل ۱۲	TS در پروژه واقعی: zod، React، API

فصل پایانی

فصل	موضوع
فصل ۱۳	چیت‌شیت دوتایی + گلاساری + نقشه راه

زمین تمرین

همه چیز با Node اجرا می‌شود (فصل ۱ نصبش می‌کنیم):

```
mkdir js-ts-playground && cd js-ts-playground
node # REPL بنویس و فوری جواب بگیر - REPL (exit با exit)
```

قانون طلایی دوره: هر مفهوم را خودت تایپ و اجرا کن – JS با خواندن یاد گرفته نمی‌شود، با شکستن و دیدن ارور یاد گرفته می‌شود. خطاها دوست تو هستند.

پیش‌نیاز

هیچ دوره از «متغیر یعنی چی؟» شروع می‌کند. (اگر دوره‌های قبلی را خوانده‌ای: CLI برای اجرای Node لازم است – فصل ۱ نصب را کامل می‌کند.)

نسخه PDF

نسخه PDF رنگی راست‌به‌چپ (فونت وزیرمتن) در همین پوشه: «دوره کامل JavaScript و TypeScript.pdf». بازسازی:

+ `python build/build_pdf.py` رندر Chrome headless

فصل ۱ – شروع: اجرای کد، متغیرها و انواع داده

🎯 هدف: نصب Node، سه راه اجرای کد JS، متغیرها (let/const – و چرا var نه)، ۷ نوع داده پایه، عملگرها و template literal. آخر فصل اولین برنامه‌های واقعی را اجرا کرده‌ای.

۱.۱ – نصب Node و سه راه اجرا

Node.js = موتور JS بیرون از مرورگر (دوره‌های قبلی نصبش کرده‌ای – `node -v`).

سه راه اجرای کد JS:

```
node          # ۱) REPL: فوری جواب بگیر
node app.js   # ۲) اجرای فایل
# ۳) برای کدهای مرتبط با صفحه – کنسول مرورگر (F12 → Console)
```

زمین تمرین: `mkdir js-ts-playground && cd js-ts-playground` – فایل‌های هر فصل اینجا.

۱.۲ – متغیرها: جعبه‌های داده

```
const name = "علی"; // ثابت – بازتخصیص ممنوع
let score = 20;     // let: قابل تغییر
score = 25;         // ✓
// name = "رضا";   // ✗ TypeError: Assignment to constant variable

var old = "فرار کن"; // مدل قدیمی – فصل ۳ توضیح می‌دهد چرا نمی‌خواهیم
```

قواعد اسم‌گذاری: حروف/عدد/\$_ / _، شروع با عدد ممنوع، (camelCase) `userName`، `data1` – معنادار – دشمن آینده‌ی توست.

💡 قاعده انتخاب: پیش‌فرض `const`. اگر جایی نیاز به تغییر دیدی، همان خط را به `let` عوض کن – کد را از تغییرهای بی‌دلیل ایمن نگه می‌دارد.

۱.۳ – انواع داده پایه (۷ نوع primitive + آبجکت)

```
// — v primitive —
const text = "سلام"; // String - رشته
const age = 25; // Number - عدد (اعشار هم همین)
const ok = true; // Boolean - درست/غلط
const nothing = undefined; // مقدار داده نشده
const empty = null; // عمداً خالی
const big = 9007199254740993n; // BigInt - اعداد خیلی بزرگ
const id = Symbol("id"); // Symbol - شناسه یکتا (کمتر لازم می‌شود)

// — آبجکت (همه چیزهای پیچیده از آن ساخته می‌شوند) —
const user = { name: "علی", age: 25 }; // Object
```

چک نوع با `typeof`:

```
typeof "سلام" // "string"
typeof 25 // "number"
typeof true // "boolean"
typeof undefined // "undefined"
typeof null // "object" ← آبجکت نیست null! JS باگ تاریخی ⚠️
typeof {} // "object"
typeof [1,2] // "object" ← آرایه هم آبجکت است
typeof (() => {}) // "function"
```

⚠️ دو تله کلاسیک: (۱) `typeof null` می‌گوید "object" — باگ ۳۰ ساله که برای سازگاری نگاه داشته شده؛ (۲) آرایه `typeof` اش object است — تشخیص آرایه با `Array.isArray(x)`.

۱.۴ – عملگرها

```
// حسابی
10 + 3 // 13
10 / 3 // 3.333... (*: تفریق: -, ضرب)
10 % 3 // 1 (n % 2 === 0) باقیمانده: تشخیص زوج/فرد ←
2 ** 10 // 1024 ← توان

// مقایسه - همیشه
5 === 5 // true (مقدار + نوع)
5 == "5" // true ⚠️ استفاده نکن!
5 !== "5" // true

// منطقی
true && false // false (و)
true || false // true (یا)
!true // false (نقیض)

// nullish - مدرن و مهم:
const port = userInput ?? 3000; // بود ۳۰۰۰ null/undefined فقط اگر
// برخلاف // که 0 و "" را هم می‌گیرد!
```

🔔 سه‌گانه‌ی باگ‌ساز مبتدی‌ها: = (انتساب) با === (مقایسه) اشتباه گرفتن؛ استفاده از ==؛ و جمع رشته با عدد: `5 + 3` می‌شود "53"! (JS رشته برنده است.) برای تبدیل واقعی: `8 → 5 + 3` . `Number("5") + 3`

۱.۵ – Template Literal: ساخت رشته مثل حرفه‌ای‌ها

```
const user = "علی";
const score = 95;

// قدیمی - پر از + و خطا ❌
console.log("گرفت " + score + " نمره " + user + " کاربر");

// مدرن - backtick (``) + ${} ✅
console.log(`گرفت ${score} نمره ${user} کاربر`);
console.log(`3 * 2 = 6 در سه برابر`);
console.log(`چند خطی`);
console.log(`هم بدون + کار می‌کند`);
```

`${}` هر عبارت JS را می‌گیرد - حتی فراخوانی تابع. از این به بعد همیشه همین.

۱.۶ – اولین برنامه واقعی

```
// grade.js - محاسبه وضعیت نمره
const student = "سارا";
const score = 78;
const passMark = 50;
const passed = score >= passMark;

console.log(`دانشجو: ${student}`);
console.log(`نمره: ${score} - وضعیت: ${passed ? "مردود" : "قبول ✅"}`);
console.log(`درصد: ${Math.round((score / 100) * 100)}%`);
```

```
$ node grade.js
دانشجو: سارا
نمره: 78 - وضعیت: قبول ✅
%درصد: 78
```

(آن ternary = "قبول" : "مردود" ? passed - مقدمه‌ی فصل ۱۲!)

جمع‌بندی فصل ✅

- سه راه اجرا: `node` (REPL)، فایل (`node app.js`)، کنسول مرورگر
- متغیر: پیش‌فرض `const`، تغییرپذیر `let`، هرگز `var`
- `primitive` + آبجکت؛ `typeof`؛ تله‌های `typeof null` و آرایه
- همیشه `??` برای nullish؛ `"5" + 3 = "53"`!
- Template literal با backtick و `${}` - استاندارد ساخت رشته

تمرین فصل ۱ (در زمین تمرین - هر تمرین یک فایل) 📄

۱. `info.js`: نام، سن، شهر خود را در متغیرها بگذار و با template literal یک جمله معرفی چاپ کن.

2. `calc.js` : دو عدد در متغیر؛ جمع/تفریق/ضرب/باقیمانده را با `template` چاپ کن.

3. حدس بزن، بعد تست کن: `typeof null` ، `typeof [1,2]` ، `typeof NaN` ، `"5" + 3` ، `Number("5") + 3`

4. `swap.js` : دو متغیر `a` و `b` را بدون متغیر سوم جابجا کن. (راهنما: `[a, b] = [b, a]` – destructuring – یا با فکر کن!)

5. چالش: تبدیل دقیقه به «ساعت و دقیقه» – «ساعت و 15 دقیقه 2» `135` → (با `%` و `Math.floor`)

▼ جوابها

```
// تمرین ۳:
typeof null // "object" - باگ تاریخی
typeof [1,2] // "object"
typeof NaN // "number" ← NaN یعنی Not-a-Number
"5" + 3 // "53" (رشته)
Number("5") + 3 // 8

// تمرین ۵:
const minutes = 135;
const h = Math.floor(minutes / 60); // 2
const m = minutes % 60; // 15
console.log(`ساعت ${h} و دقیقه ${m}`);
```

→ فصل بعد: تصمیم‌گیری و تکرار – شرطها، حلقه‌ها و توابع.

فصل ۲ – کنترل جریان: شرطها، حلقه‌ها و توابع پایه

🎯 هدف: مغز برنامه – تصمیم‌گیری (if/switch/ternary)، تکرار (for/while/for-of) و بسته‌بندی منطق در تابع. آخر فصل می‌توانی منطق‌های واقعی بنویسی.

۲.۱ – if / else if / else

```
const score = 78;

if (score ≥ 90) {
  console.log("عالی 🏆");
} else if (score ≥ 70) {
  console.log("خوب 👍");
} else if (score ≥ 50) {
  console.log("قبول");
} else {
  console.log("مردود");
}
```

- شرط همیشه به boolean تبدیل می‌شود – و مقادیر «falsy» وجود دارند که false حساب می‌شوند:

```
// falsy: false, 0, "", null, undefined, NaN
// "truthy 0" اند: "false", [], {}

if ( "") console.log("اجرا نمی‌شود"); // رشته خالی = falsy
if ( []) console.log("اجرا می‌شود");  // آرایه خالی = truthy! ⚠️
```

⚠️ تله کلاسیک: `if (value)` برای «وجود دارد» – وقتی `value` می‌تواند 0 باشد خراب می‌شود (`!falsy = 0`). برای «null/undefined» دقیق‌تر: `if (value ≠ null)` یا `if (value ≇ undefined)`.

```
// ternary - اگر غلط : شرط یک خطی: شرط ? اگر درست :
const status = score ≥ 50 ? "قبول" : "مردود";
const label = score ≥ 90 ? "عالی" : score ≥ 70 ? "زنجیره" // "معمولی";

// switch - وقتی یک مقدار را با چند حالت ثابت مقایسه می‌کنی
const day = "sat";
switch (day) {
  case "sat":
  case "sun":
    console.log("آغاز هفته");
    break; // ⚠️ بدون break به case بعدی می‌ریزد (fall-through)!
  case "fri":
    console.log("تعطیل");
    break;
  default:
    console.log("روز عادی");
}
```

قاعده انتخاب: ۲-۳ حالت → ternary/if؛ ۴+ حالت ثابت → switch (یا آبجکت lookup – فصل ۴!).

۲.۳ – حلقه‌ها

```
// for - کلاسیک - وقتی شمارنده می‌خواهی
for (let i = 1; i ≤ 5; i++) {
  console.log(`ردیف ${i}`);
}

// while - وقتی تعداد تکرار نامشخص است
let attempts = 0;
while (attempts < 3) {
  console.log(`تلاش ${attempts + 1}`);
  attempts++; // ⚠️ بدون این: حلقه بی‌نهایت (Ctrl+C در ترمینال)
}

// for...of - پیمایش آرایه (مدرن و خوانا) ★
const skills = ["JS", "TS", "React"];
for (const skill of skills) {
  console.log(`در حال یادگیری ${skill}`);
}

// break و continue
for (const n of [1, 2, 3, 4, 5]) {
  if (n === 3) continue; // این دور را رد کن
  if (n === 5) break; // از حلقه خارج شو
  console.log(n); // 1 2 4
}
```

💡 برای آرایه‌ها معمولاً `for...of` یا متدها (`map/filter` – فصل ۴) انتخاب‌اند؛ `for` کلاسیک برای شمارنده‌های واقعی (۱ تا ۱۰) یا پیمایش با ایندکس خاص.

۲.۴ – توابع: بسته‌بندی منطق

```
// Function Declaration – با اسم
function greet(name) {
  return `سلام ${name}!`;
}

// فراخوانی:
const msg = greet("علی"); // "سلام علی"
```

- `return` = خروجی تابع؛ بعد از `return` اجرای تابع تمام می‌شود
- تابع بدون `undefined` → `return` برمی‌گرداند
- پارامتر پیش‌فرض:

```
function greet(name = "مهمان") {
  return `سلام ${name}!`;
}
greet(); // "سلام مهمان"
```

- تابع به عنوان مقدار: توابع در JS «اول‌رده» اند – می‌توانند در متغیر، آرگومان و خروجی باشند:

```
const double = function (n) { return n * 2 }; // function expression
const triple = (n) => n * 3; // ★ arrow function – فصل ۳ کامل

function applyTwice(fn, value) {
  return fn(fn(value)); // تابع را به عنوان آرگومان پاس می‌دهیم!
}
applyTwice(double, 5); // 20
```

۲.۵ – برنامه واقعی: بازی حدس عدد

```
// guess.js – چرخه کامل شرط/حلقه/تابع
function checkGuess(guess, secret) {
  if (guess === secret) return "🎉! درست بود";
  if (guess < secret) return "⬆️! بزرگ‌تر بگو";
  return "⬆️! کوچک‌تر بگو";
}

const secret = Math.floor(Math.random() * 100) + 1; // تا ۱۰۰
let guess = 50;

// !خودت بازی کن (در REPL) شبیه‌سازی ۵ تلاش
for (let i = 1; i <= 5; i++) {
  console.log(`تلاش ${i}: ${guess} → ${checkGuess(guess, secret)}`);
  guess = guess < secret ? guess + 7 : guess - 3; // استراتژی ساده
}
```

جمع‌بندی فصل ✓

- falsy ها: `false/0/"/null/undefined/NaN` – و تلهی `if (value)` با عدد صفر

- ternary برای یک خط، switch برای حالت‌های ثابت (break یادت نره)
- for (شمارنده) / while (تا وقتی) / for...of (پیمایش آرایه) + break/continue
- تابع = بسته‌بندی منطق؛ return خروجی؛ پیش‌فرض پارامتر؛ توابع مقادیرند
- ترکیب‌شان = هر منطقی که در پروژه لازم داری

تمرین فصل ۲

1. `fizzbuzz(n)`: اعداد ۱ تا n – مضرب ۳ «Fizz»، مضرب ۵ «Buzz»، مضرب هر دو «FizzBuzz»، بقیه خود عدد. (مسئله‌ی افسانه‌ای مصاحبه!)
2. `max3(a, b, c)`: بزرگ‌ترین سه عدد را برگرداند – دو راه: با if و با ternary.
3. حلقه‌ای بنویس جدول ضرب ۷ را چاپ کند: `7 × 1 = 7 ... 7 × 10 = 70`
4. `isEven(n)` بنویس بدون % (راهنما: `& 1` bitwise یا چک روی رقم آخر رشته)
5. چالش: تابع `countVowels(text)` – تعداد حروف صدا دار (a,e,i,o,u – فارسی: ا،و،ی) را بشمارد.

```
// ۱) FizzBuzz
function fizzbuzz(n) {
  for (let i = 1; i ≤ n; i++) {
    if (i % 15 === 0) console.log("FizzBuzz");
    else if (i % 3 === 0) console.log("Fizz");
    else if (i % 5 === 0) console.log("Buzz");
    else console.log(i);
  }
}
// فقط می‌گیرد Fizz نکته: اول % 15 (هر دو) - وگرنه

// ۲) max3
function max3(a, b, c) {
  if (a ≥ b && a ≥ c) return a;
  return b ≥ c ? b : c;
}
// ساده‌تر Math.max(a, b, c) با Math.max

// ۳)
for (let i = 1; i ≤ 10; i++) console.log(`7 × ${i} = ${7 * i}`);

// ۴)
function isEven(n) {
  return (n & 1) === 0; // بیت آخر صفر = زوج
  // یا: String(n).at(-1) === "0" در "02468".includes(...)
}

// ۵)
function countVowels(text) {
  let count = 0;
  for (const ch of text.toLowerCase()) {
    if ("aeiou".includes(ch)) count++;
  }
  return count;
}
```

→ فصل بعد: توابع عمیق — arrow، closure، scope و callback.

فصل ۳ – توابع عمیق: Arrow، Closure، Scope و Callback

🎯 هدف: مرز جونیور و متوسط در JS همین جاست. چهار مفهوم: scope (دید متغیرها)، hoisting، closure (بستار – قلب JS)، و arrow function ها. این فصل را دو بار بخوان – بار دوم همه چیز جا می افتد!

۳.۱ – Scope: متغیر کجا «زندگی» می کند؟

```
const global = "همه جا دیدنی"; // سراسری scope

function outer() {
  const outerVar = "و فرزندانش outer دیدنی در";

  function inner() {
    const innerVar = "فقط همین جا";
    console.log(global); // ✅ بالا سرش را می بیند
    console.log(outerVar); // ✅ والد
    console.log(innerVar); // ✅ خودش
  }
  inner();
  // console.log(innerVar); // ❌ ReferenceError - از scope خارج
}
```

قاعده: تابع فرزند به متغیرهای والد دسترسی دارد، برعکس نه. جستجوی متغیر از داخل به بیرون (scope chain) انجام می شود.

let/const با Block scope

```
{
  const inside = "فقط همین بلوک";
}
// console.log(inside); // ❌ خارج از بلوک

if (true) {
  let blockLet = "هم بلوکی است let";
}
```

پس var چه دردسری دارد؟

```
// است - بلوک را نمی شناسد function-scoped فقط var
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i)); // 3 3 3 ← همان i هر سه بعد از حلقه، همان
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j)); // 0 1 2 ✅ خودش j، هر دور
}
```

- `var`: hoisting با `undefined` بالا می‌آید، `let/const` تا تعریف TDZ دارند. نتیجه: همیشه `let/const`، هرگز `var`.

۳.۲: Hoisting – بالا آمدن تعریف‌ها

```
sayHi();           //  کار می‌کند! - declaration hoist کامل می‌شود
function sayHi() { console.log("hi"); }

console.log(x);    // undefined - var hoist می‌شود ولی مقدار نه
var x = 5;

console.log(y);    //  ReferenceError - let/const در TDZ اند
let y = 5;
```

۳.۳: Closure – بستار (مفهوم طلایی JS)

تعریف: وقتی یک تابع از درون تابع دیگر ساخته می‌شود، به متغیرهای تابع پدر – حتی بعد از پایان اجرای پدر – دسترسی دارد. «تابع + حافظه‌اش»:

```
function makeCounter() {
  let count = 0;           // متغیر خصوصی - از بیرون دسترسی نیست
  return function () {    // closure: count دارد یاد را به
    count++;               //
    return count;
  };
}

const counter1 = makeCounter();
const counter2 = makeCounter(); // مستقل دارند counter هر کدام

counter1(); // 1
counter1(); // 2
counter2(); // 1 ← مستقل!
```

سه کاربرد واقعی:

```
// ۱) class داده خصوصی - بدون
function makeWallet(initial) {
  let balance = initial;
  return {
    deposit: (n) => (balance += n),
    getBalance: () => balance,
  };
}

// ۲) debounce/throttle (در مصاحبه‌ها و پروژه‌ها)
function debounce(fn, delay) {
  let timer; // زنده است closure
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// ۳) پیکربندی تابع
const multiplier = (factor) => (n) => n * factor;
const byTwo = multiplier(2);
byTwo(5); // 10
```

💡 بگو: «closure یعنی تابع + محیطی که در آن متولد شد» – همین یک جمله در مصاحبه کافی است.

۳.۴ – Arrow Function: خلاصه، lexical this

```
// از طولانی به کوتاه:
const add = (a, b) => { return a + b; };
const add = (a, b) => a + b; // ضمنی return: بدنه تک‌عبارتی
const square = n => n * n; // تک‌پارامتر: بدون پرانتز
const zero = () => 0;

// (های آرایه callback) خواندن کد دیگران:
const names = users.map(u => u.name).filter(n => n.length > 3);
```

فرق اصلی با function معمولی: **this** arrow خودش را ندارد – از محیط اطراف می‌گیرد (lexical this). در کلاس‌ها و event handler های قدیمی حیاتی بود؛ در function components مدرن فرقی کم‌تر حس می‌شود ولی استایل استاندارد کد مدرن است.

۳.۵ – Callback: تابع به عنوان آرگومان

```
function processUser(user, onSuccess, onError) {
  if (user.age ≥ 18) onSuccess(user);
  else onError(new Error("۱۸ زیر"));
}

processUser(
  { name: "علی", age: 20 },
  (u) => console.log(`خوش آمدی ${u.name}`),
  (err) => console.error(err.message)
);
```

callback پایه‌ی همه‌چیز است: متدهای آرایه (map/filter – فصل ۴)، event listener ها، و حتی Promise ها (فصل ۷ – که callback hell را هم حل کردند!).

🚨 **Callback Hell** – قبل از Promise ها، async های تو در تو مثل هرم آبی می‌شدند:

```
getUser(id, (user) => {
  getOrders(user, (orders) => {
    getDetails(orders[0], (details) => {
      // «تودرتو و غیرقابل مدیریت – «هرم آبی ...»
    });
  });
});
```

علاجش async/await است – فصل ۷.

✅ جمع‌بندی فصل

- scope: فرزند والد را می‌بیند، برعکس نه؛ let/const بلوکی، var نه (و باگ setTimeout)
- hoisting: function declaration کامل، var بی‌مقدار، let/const در TDZ
- **closure** = تابع + حافظه‌اش: counter، داده خصوصی، debounce
- arrow: خلاصه + return ضمنی + lexical this
- callback = تابع به عنوان آرگومان – پایه‌ی async و متدهای فصل ۴

📝 تمرین فصل ۳

1. خروجی را حدس بزن (قبل از اجرا!):

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var:", i));
}
```

2. `makePasswordChecker(minLength)` بنویس که تابعی برگرداند: رشته بگیرد و true/false بدهد – با closure (هر

checker حد خودش را دارد).

3. خروجی؟

```
const obj = {
  name: "علی",
  say: function () { console.log(this.name); },
  sayArrow: () => console.log(this.name),
};
obj.say(); // ?
obj.sayArrow(); // ?
```

4. با callback ها: تابع `repeat(n, action)` بنویس که action را n بار صدا بزند. تست: `repeat(3, (i) =>`

`console.log("دور", i))`

5. چالش: با closure یک `makeCache()` بنویس که `get(key)` و `set(key, value)` بدهد و داده را بین فراخوانی‌ها حفظ کند.

▼ جواب‌ها

1. سه بار «var» – «var: 3» بلوکی نیست؛ سه callback بعد از پایان حلقه اجرا می‌شوند و همه همان i نهایی (3) را می‌بینند. با `let: 0, 1, 2`.

2.

```
function makePasswordChecker(minLength) {  
  return (password) => password.length >= minLength;  
}  
const check8 = makePasswordChecker(8);  
check8("abc"); // false
```

3. اولی «علی» (`this = obj`)؛ دومی `undefined` (یا `global` – چون `this` arrow ندارد و از `scope` بیرونی می‌گیرد که `obj` نیست!). سوال محبوب مصاحبه.

4.

```
function repeat(n, action) {  
  for (let i = 0; i < n; i++) action(i);  
}
```

5.

```
function makeCache() {  
  const store = new Map(); // زنده می‌ماند closure در  
  return {  
    get: (key) => store.get(key),  
    set: (key, value) => store.set(key, value),  
  };  
}
```

➔ فصل بعد: آبجکت‌ها و آرایه‌ها – پرکاربردترین ساختارها و متدهایشان.

فصل ۴ – آبجکت‌ها و آرایه‌ها: پرکاربردترین ساختارهای JS

🎯 هدف: ۹۰٪ داده‌ی هر اپ JS با این دو ساختار جابه‌جا می‌شود (JSON = همین‌ها!). ساخت، دسترسی، destructuring، spread و متدهای آرایه – قلب کار روزمره‌ات.

۴.۱ – آبجکت: داده ساخت یافته

```
const user = {  
  name: "علی", // key: value  
  age: 25,  
  "favorite lang": "JS", // کلید با فاصله → کوتیشن لازم  
  email: "ali@dev.ir",  
};  
  
// دسترسی:  
user.name // "علی" (نقطه‌ای - رایج)  
user["favorite lang"] // وقتی کلید فاصله/متغیر است  
  
// تغییر و افزودن:  
user.age = 26;  
user.city = "تهران"; // کلید جدید - همین‌قدر ساده  
delete user.email;  
  
// کلید از متغیر:  
const key = "name";  
user[key]; // "علی" (براکتی با متغیر)  
  
// چک وجود:  
"name" in user // true  
user.phone ?? "ثبت نشده" // undefined → (فصل ۱)  
  
// پیمایش:  
for (const key in user) {  
  console.log(key, "→", user[key]);  
}  
  
Object.keys(user); // ["name", "age", "city", ...]  
Object.values(user); // مقادیر  
Object.entries(user); // map عالی برای - ... ["name", "علی"], ...
```

متد داخل آبجکت:

```
const counter = {  
  count: 0,  
  increment() { // shorthand متد  
    this.count++; // this = خود آبجکت  
  },  
};  
counter.increment();  
counter.count; // 1
```

```
const skills = ["JS", "TS", "React"];

skills[0];           // "JS"      (!!ایندکس از صفر)
skills.length;       // 3
skills.at(-1);       // "React" ← آخرین عنصر (at منفی ایندکس منفی)
skills.push("CSS");   // افزودن آخر
skills.pop();         // حذف آخر
skills.unshift("HTML"); // افزودن اول
skills.shift();       // حذف اول
skills.includes("JS"); // true
skills.indexOf("TS"); // 1- یا اگر نیست) 1
skills.slice(1, 3);   // برش بدون تغییر اصلی ["TS", "React"]
skills.join(" * ");   // "JS * TS * React"
```

۴.۳ – destructuring: باز کردن بسته‌ها ★

استخراج تمیز – همه‌جای کد مدرن:

```
// آبجکت:
const user = { name: "علی", age: 25, city: "تهران" };
const { name, age, city } = user; // با پیش‌فرض
// name="علی", age=25, city="تهران"

// تغییر نام:
const { name: fullName } = user; // fullName = "علی"

// تودرتو:
const response = { data: { user: { email: "a@b.c" } } };
const { data: { user: { email } } } = response; // email = "a@b.c"

// آرایه (بر اساس ترتیب):
const [first, second] = skills; // "JS", "TS"
const [head, ...rest] = skills; // rest = بقیه

// در پارامتر تابع (شخصی‌ترین کاربرد):
function printUser({ name, age }) {
  console.log(`${name} - ${age}`);
}
printUser(user); // فقط آبجکت می‌فرستی
```

۴.۴ – spread و ... rest

```
// spread - immutable! باز کردن: کپی و ترکیب
const updated = { ...user, age: 27 }; // کپی + تغییر یک فیلد
const merged = { ...defaults, ...overrides }; // بعدی بر قبلی غلبه می‌کند
const all = [...arr1, ...arr2]; // ترکیب آرایه‌ها

// rest - جمع کردن: پارامترهای نامشخص
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4); // 10
```

⚠ یادآوری مهم: `{ user }`. یک `shallow copy` است – آبجکت‌های تودرتو هنوز مرجع مشترک دارند (فصل ۵ TS و `deep copy` را هم ببین).

۴.۵ – متدهای آرایه: `map / filter / find / reduce` ★★

مهم‌ترین بخش کل دوره – در هر پروژه React هر روز این‌ها را می‌نویسی:

```
const products = [
  { id: 1, title: "گوشی", price: 50, category: "mobile", inStock: true },
  { id: 2, title: "لپ‌تاپ", price: 120, category: "laptop", inStock: false },
  { id: 3, title: "ایرپاد", price: 12, category: "mobile", inStock: true },
];

// map - تبدیل هر عنصر (همیشه هم‌طول)
const titles = products.map(p => p.title);

// filter - فقط عناصری که شرط دارند
const available = products.filter(p => p.inStock);

// find - اولین عنصر منطبق (یا undefined)
const phone = products.find(p => p.id === 1);

// some / every - «همه» / «حداقل یکی»
products.some(p => p.price > 100); // true
products.every(p => p.inStock); // false

// sort - مرتب‌سازی (⚠ می‌شوند مگر مقایسه بدهی sort اعداد به رشته)
[...products].sort((a, b) => a.price - b.price); // روی کپی! صعودی - روی کپی

// reduce - تاخوردن به یک مقدار (جمع، گروه‌بندی، شمارش)
const total = products.reduce((sum, p) => sum + p.price, 0); // 182

// زنجیره‌سازی - قدرت واقعی:
const mobileTitles = products
  .filter(p => p.category === "mobile")
  .map(p => `${p.title} (${p.price}M)`);
// ["گوشی (50M)", "ایرپاد (12M)"]
```

سه قانون طلایی: 1. `map` برای تبدیل (هم‌طول)، `filter` برای حذف، `reduce` برای تاخوردن – درست انتخاب کن 2. این‌ها آرایه جدید می‌سازند – اصل را نمی‌شکنند (immutable فلسفه React!) 3. زنجیره را می‌شود شکست: هر متد، ورودی متد بعدی

groupBy با reduce (سوال مصاحبه!)

```
const byCategory = products.reduce((acc, p) => {
  (acc[p.category] ??= []).push(p);
  return acc;
}, {});
// { mobile: [...], laptop: [...] }
```

جمع‌بندی فصل ✓

- آبجکت: نقطه‌ای/براکتی، this، Object.keys/values/entries، in/?? در متد
- آرایه: at(-1)، push/pop، slice/join/includes
- destructuring (با پیش‌فرض و تو در تو) + در پارامترها – استایل مدرن
- spread برای کپی/ترکیب، rest برای جمع‌کردن – shallow copy حواست هست!
- map/filter/find/reduce = قلب کد فرانت؛ immutable و زنجیره‌پذیر

تمرین فصل ۴ 📄

1. از آرایه products بالا: (الف) فقط عنوان‌های موجودها (ب) گران‌ترین محصول (ج) میانگین قیمت – هرکدام یک زنجیره.
2. `pluck(items, key)` بنویس: آرایه آبجکت + نام کلید → آرایه مقادیر آن کلید. (`pluck(products, "title")`)
3. destructuring تمرین: از `const meta = { info: { title: "t", tags: ["a","b"] } }` تگ اول و دوم را یک خطی استخراج کن.
4. چرا این کد باگ دارد؟ `const copy = products; copy.push(newItem);` – درستش کن.
5. چالش: تابع `sumBy(items, keyFn)` با `reduce` بنویس – جمع یک فیلد دلخواه.

▼ جواب‌ها

```
// ۱
products.filter(p => p.inStock).map(p => p.title);
[...products].sort((a, b) => b.price - a.price)[0];
products.reduce((s, p) => s + p.price, 0) / products.length;

// ۲
function pluck(items, key) {
  return items.map((item) => item[key]);
}

// ۳
const { info: { tags: [firstTag, secondTag] } } = meta;

// ۴: copy هر دو اثر می‌گذارد - است products همان مرجع - push
// درست: const copy = [...products]; copy.push(newItem);

// ۵
function sumBy(items, keyFn) {
  return items.reduce((sum, item) => sum + keyFn(item), 0);
}
sumBy(products, (p) => p.price);
```

➡ فصل بعد: رشته‌ها، اعداد، Date و JSON – ابزارهای داده.

فصل ۵ – رشته‌ها، اعداد، Date و JSON

🎯 هدف: ابزارهای کار با داده: متدهای پرکاربرد String، تله‌های Number (و Intl برای فارسی!)، Date مدرن و JSON – زبان تبادل داده‌ی همه API ها.

۵.۱ – رشته‌ها (String)

```
const s = " Hello JavaScript ";

// پاکسازی و بررسی:
s.trim();           // "Hello JavaScript" - حذف فاصله‌های دو طرف
s.trim().length;    // 17
s.toUpperCase();    // همه بزرگ
"ali@dev.ir".includes("@") // true - شامل است؟
s.startsWith(" He"); // true
"report.pdf".endsWith(".pdf"); // true

// برش و جستجو:
"2026-09-13".split("-"); // ["2026","09","13"] - split به آرایه
["a","b"].join(", ");    // "a, b" - split برعکس
"hello world".slice(0, 5); // "hello" (برش با ایندکس)
"hello".replace("l", "L"); // "heLlo" - فقط اولین!
"hello".replaceAll("l", "L"); // "heLLo" - همه
"a,b,c".replaceAll(",", " | ");

// و تبدیل (فصل ۱) template ترکیب با:
Number("42");        // 42
String(42);           // "42"
parseInt("42px");     // 42 (تا جایی که عدد است)
parseFloat("3.14x"); // 3.14
```

⚠ رشته‌ها در JS immutable اند: `s.trim()` رشته‌ی جدید می‌دهد؛ `s` بدون تغییر می‌ماند! پس نتیجه را

ذخیره کن: `const clean = s.trim();`

۵.۲ – اعداد (Number) و Intl: نمایش فارسی ★

```
// !دقت اعشاری - تله مشهور
0.1 + 0.2 // 0.30000000000000004
+(0.1 + 0.2).toFixed(2) // 0.3 ← عدد صحیح نگه دار ←

// تبدیل‌ها:
parseInt("42"); // 42
Number("42"); // 42
Number("abc"); // NaN ← Not a Number (نوعش!)
Number.isNaN(NaN); // true ← فقط با NaN چک ←

// Intl (JS! جواهر پنهان) فرمت‌بندی به زبان/منطقه ★
const price = 1250000;
new Intl.NumberFormat("fa-IR").format(price); // "۱,۲۵۰,۰۰۰"
new Intl.NumberFormat("fa-IR", {
  style: "currency", currency: "IRR",
}).format(price); // "۱,۲۵۰,۰۰۰ ریال"

new Intl.DateTimeFormat("fa-IR").format(new Date()); // "۱۴۰۵/۶/۲۲"
new Intl.ListFormat("fa-IR").format(["سارا", "علی"]); // "علی و سارا"
```

دقیقاً همین `toLocaleString("fa-IR")` را در پروژه Next خودت هم دیدی – Intl زیرش است.)

۵.۳ – Date: کار با تاریخ

```
const now = new Date(); // الان
const bd = new Date("1995-06-15"); // ISO از رشته

now.getFullYear(); now.getMonth(); // تا ۱۱! ژانویه ۰ ۰ = ⚠️
now.getDate(); now.getDay(); // روز هفته (یکشنبه=۰) / ماه / روز

// محاسبه فاصله (با میلی‌ثانیه):
const diffMs = now - bd; // JS می‌کند!
const diffDays = Math.floor(diffMs / 86400000);

// فرمت فارسی:
new Intl.DateTimeFormat("fa-IR", {
  dateStyle: "full",
  timeStyle: "short",
}).format(now); // "یکشنبه ۲۲ شهریور ۱۴۰۵، ۱۹:۴۵"

// setTimeout/sleep با Date (پایه async و فصل ۷):
const start = Date.now();
Date.now() - start; // گذشت ms چند
```

⚠️ در اپ واقعی (منطقه‌زمانی‌ها و محاسبات پیچیده)، از کتابخانه `date-fns` یا `dayjs` استفاده کن – `Date` بومی برای پایه کافی است ولی تاریخ در production بغرنج است.

```
const user = { name: "علی", age: 25, skills: ["JS", "TS"] };

// آبجکت → رشته (برای ارسال/ذخیره):
const json = JSON.stringify(user);
// '{"name": "علی", "age": 25, "skills": ["JS", "TS"]}'

// با فرمت خوانا (برای دیباگ):
JSON.stringify(user, null, 2);

// رشته → آبجکت (پاسخ API):
const parsed = JSON.parse(json);
parsed.name; // "علی"

// ⚠️ JSON.parse می‌کند - همیشه throw روی رشته نامعتبر
try {
  const data = JSON.parse(rawString);
} catch {
  console.error("JSON خراب است");
}
```

قوانین JSON: کلیدها حتماً با `"` و `.` دوتایی؛ کامنت ندارد؛ تابع/undefined نمی‌پذیرد. (چرخه کامل: در دوره‌ی Prisma `JSON.stringify` می‌رفت سمت API و `JSON.parse` برمی‌گشت – حالا زیرش را می‌دانی!)

جمع‌بندی فصل ✓

- رشته: `trim/split/join/replace/include` – immutable؛ نتیجه را ذخیره کن
- اعداد: `toFixed` برای نمایش، `Intl` برای فارسی (`NumberFormat/DateTimeFormat`)
- `Date: getMonth` از صفر! اختلاف = میلی‌ثانیه؛ پیچیده شد → `dayjs/date-fns`
- JSON: `stringify/parse` – زبان مشترک API؛ `parse` با `try/catch`

تمرین فصل ۵ 📝

1. `slugify(title)`: «مقدمه‌ای بر JS!» → رشته لاتین تمیز با `-` (راهنما: `trim`، `lowercase`، `replaceAll`؛ فاصله به `-`).
2. قیمت `1234567` را به سه شکل چاپ کن: با جداکننده فارسی، با «تومان»، و `toFixed(2)`.
3. سن از تاریخ تولد: `getAge("1995-06-15")` – سال کامل حساب‌شده (نه فقط اختلاف سال!).
4. یک آبجکت بساز، `stringify` کن، دوباره `parse` کن و تفاوت دو آبجکت را با `JSON.stringify(a) === JSON.stringify(b)` چک کن (و نکته‌اش را بگو: مقایسه عمیق با این روش شکننده است – ترتیب کلیدها مهم است!).
5. چالش: `formatRelative(days)` – «امروز»، «دیروز»، «۳ روز پیش»، «۳۰ روز پیش».

```
// ۱
function slugify(title) {
    return title.trim().toLowerCase().replaceAll(/\s+/g, "-");
}
// (regex \s+ = فصل ۱ آشناتر می‌شود یا چند فاصله - فصل ۱ آشناتر می‌شود)

// ۲
new Intl.NumberFormat("fa-IR").format(1234567); // ۱,۲۳۴,۵۶۷
new Intl.NumberFormat("fa-IR").format(1234567) + " تومان";
Number(1234567).toFixed(2); // "1234567.00"

// ۳
function getAge(birthDateString) {
    const birth = new Date(birthDateString);
    const today = new Date();
    let age = today.getFullYear() - birth.getFullYear();
    const beforeBirthday =
        today.getMonth() < birth.getMonth() ||
        (today.getMonth() === birth.getMonth() && today.getDate() < birth.getDate());
    if (beforeBirthday) age--;
    return age;
}

// ۵
function formatRelative(days) {
    if (days === 0) return "امروز";
    if (days === 1) return "دیروز";
    if (days === -1) return "فردا";
    return days > 0 ? `${days} روز پیش` : `${-days} روز بعد`;
}
```

➔ فصل بعد: DOM و رویدادها — JS را به صفحه وصل می‌کنیم!

فصل ۶ – DOM و رویدادها: زنده کردن صفحه

🎯 هدف: JS بدون DOM فقط محاسبه است – با DOM صفحه ساخته می‌شود. انتخاب عناصر، تغییر متن / استایل، ساخت عنصر و مدیریت رویداد. (در React این کارها خودکار می‌شوند – ولی برای فهم React و پروژه‌های vanilla، این پایه‌ست.)

۶.۱ – انتخاب عناصر

یک فایل `index.html` بساز:

```
<!DOCTYPE html>
<html lang="fa" dir="rtl">
<body>
  <h1 id="title">سلام!</h1>
  <p class="msg">پیام اول</p>
  <p class="msg">پیام دوم</p>
  <button id="btn">کلیک کن</button>
  <ul id="list"></ul>
  <script src="app.js"></script> ← تا عناصر موجود باشند body آخر
</body>
</html>
```

```
// انتخاب:
const title = document.getElementById("title");
const msgs = document.querySelectorAll(".msg"); // NodeList همه .msg ها
const btn = document.querySelector("#btn"); // مدرن‌ترین - CSS سلکتور

// خواندن و نوشتن:
title.textContent = "DOM! سلام دنیای"; // متن
title.innerHTML = "سلام! <strong>دنیای</strong>"; // HTML (⚠ فقط داده معتمد)

// استایل و کلاس:
title.style.color = "crimson";
title.classList.add("active");
title.classList.toggle("hidden"); // اگر بود بردار، نبود بگذار

// attribute ها:
btn.setAttribute("disabled", "true");
btn.getAttribute("id");
```

⚠ `innerHTML` با داده‌ی کاربر = دروازه XSS (تزریق اسکریپت)! برای متن همیشه `textContent` .

```
const list = document.querySelector("#list");

const li = document.createElement("li");
li.textContent = "یادگیری DOM";
li.classList.add("item");
list.appendChild(li); // به انتها اضافه شو

// حذف:
li.remove();
```

```
const btn = document.querySelector("#btn");

btn.addEventListener("click", (event) => {
  console.log("کلیک شد", event.target); // event = اطلاعات رویداد
  title.textContent = "کلیک شد";
});

// رویدادهای پرکاربرد:
// click, input (تایپ زنده), change (تغییر مقدار), submit (فرم)
// keydown, mouseenter, scroll, load

// جلوگیری از رفرش صفحه (preventDefault فرم - با):
const form = document.querySelector("#contact");
form.addEventListener("submit", (e) => {
  e.preventDefault(); // می‌شود refresh بدون این: صفحه
  const name = form.elements.name.value;
  console.log("ارسال:", name);
});

// ورودی زنده:
const search = document.querySelector("#search");
search.addEventListener("input", (e) => {
  console.log("تایپ شد:", e.target.value); // فصل ۳ - debounce - هر کلید
});
```

این دقیقاً همان چیزی است که React برایت خودکار می‌کند: `<button onClick={...}>` زیرش همین `addEventListener` است. فهم این فصل = فهم اینکه React چه دردی را حل می‌کند.

```
const form = document.querySelector("#todo-form");
const input = document.querySelector("#todo-input");
const list = document.querySelector("#todo-list");

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const text = input.value.trim();
  if (!text) return; // خالی؟ هیچی

  const li = document.createElement("li");
  li.textContent = text;
  li.addEventListener("click", () => {
    li.classList.toggle("done"); // کلیک روی آیتم = انجام شده
  });

  list.appendChild(li);
  input.value = ""; // پاک کردن input
});
```

همین TODO را در فصل ۸ مرور Next با React/Server Action دیدی – مقایسه کن: React با DOM، state را خودش sync می‌کند؛ اینجا تو دستی می‌سازی.

جمع‌بندی فصل ✓

- انتخاب: `querySelector` (سلکتور CSS) + `getElementById`
- `textContent` برای متن (امن)؛ `innerHTML` فقط با داده معتمد
- `classList.toggle` ؛ `createElement` + `appendChild`
- `addEventListener` + `preventDefault` در فرم‌ها؛ رویداد `input` زنده است
- React همین درد را خودکار حل می‌کند – الان می‌فهمی چرا محبوب است!

تمرین فصل ۶ 📝

1. صفحه‌ای با یک `input` و یک `<h2>`: هر چه تایپ می‌شود فوری در `h2` نمایش داده شود (رویداد `input`).
2. دکمه‌ای بساز که با هر کلیک یک عدد را یکی زیاد کند و در خودش نشان دهد (closure/state با متغیر بیرونی).
3. لیست رنگ‌ها در JS: با `for...of` برای هر رنگ یک `<li>` با `background` همان رنگ بساز.
4. فرمی با چک طول متن: زیر ۳ حرف، دکمه `submit` غیرفعال شود (`disabled`).
5. چالش: دکمه «تم» – با هر کلیک `dark` کلاس روی `toggle <body>` شود و متن دکمه عوض شود.

```
// ۱
const input = document.querySelector("#name-input");
const preview = document.querySelector("#preview");
input.addEventListener("input", (e) => {
  preview.textContent = e.target.value;
});

// ۲
let count = 0;
const btn = document.querySelector("#counter");
btn.addEventListener("click", () => {
  count++;
  btn.textContent = `کلیک‌ها: ${count}`;
});

// ۵
const themeBtn = document.querySelector("#theme-btn");
themeBtn.addEventListener("click", () => {
  const dark = document.body.classList.toggle("dark");
  themeBtn.textContent = dark ? "🌞 تاریک" : "🌙 روشن";
});
```

➔ فصل بعد: Async – بزرگ‌ترین و مهم‌ترین فصل JS: fetch، Promise و async/await.

فصل ۷ – Async کامل: Promise، fetch و async/await

🎯 هدف: مهم‌ترین فصل ۹۰٪ – JS کار دلوپر مرورگر منتظر ماندن است: fetch از سرور، تایمر، فایل. سه نسل (callback → Promise → async/await)، event loop و الگوهای واقعی.

۷.۱ – چرا async؟ و Callback

JS single-threaded است – یک کار در آن لحظه. اگر fetch سرور (۲ ثانیه) بلاک می‌کند، کل صفحه فریز می‌شود! پس عملیات طولانی **ناهمگام** (async) اجرا می‌شوند: «شروع کن، ادامه بده، وقتی حاضر شد مرا صدا بزن».

```
// فصل ۳ دیدی: callback: نسل ۱
getUser(1, (user) => {
  console.log(user);
  getOrders(user, (orders) => {
    getDetails(orders[0], (details) => {
      // 🦋 callback hell - مدیریت خطا فاجعه
    });
  });
});
```

۷.۲ – Promise: قول نتیجه آینده

Promise = رسیدی برای نتیجه‌ی عملیات async – سه وضعیت: **pending** (در راه) → **fulfilled** (رسید ✓) یا **rejected** (شکست ✗) – و بعدش قفل می‌شود.

```
const p = fetch("https://api.example.com/user/1");
// p است - هنوز داده ندارد Promise فوراً یک p

// مصرف:
p.then((response) => response.json()) // then: وقتی موفق شد
  .then((data) => console.log(data))
  .catch((err) => console.error("شکست:", err)) // catch: خطا
  .finally(() => console.log("تمام شد")); // finally: در هر صورت

// دستی Promise ساخت:
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
wait(1000).then(() => console.log("یک ثانیه گذشت"));

// ساخت خطا:
const fail = Promise.reject(new Error("اوپس"));
fail.catch((e) => console.error(e.message));
```

۷.۳ – async/await: خوانایی همگام ★

شیرین‌سازی Promise ها – کد async که مثل sync خوانده می‌شود:

```

async function loadUser(id) {
  try {
    const res = await fetch(`https://api.example.com/user/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch 404 روی throw نمی‌کند
    const user = await res.json();
    return user;
  } catch (err) {
    console.error("خطا:", err.message);
    return null;
  } finally {
    console.log("تمام شد");
  }
}

```

قوانین: - `await` فقط داخل تابع `async` (یا top-level در ماژول‌ها) - `await` فقط Promise ها را متوقف «منطقی» می‌کند (thread بلاک نمی‌شود!) - خطای `await` شده = `throw` با `try/catch` تابع `async` همیشه Promise برمی‌گرداند - حتی بدون `await`

★ Waterfall در برابر موازی

```

// ❌ ۵۰۰ میلی‌ثانیه: سریالی: ۳۰۰ms + ۲۰۰ms = ۵۰۰ms
const user = await getUser();
const posts = await getPosts();

// ✅ ۵۰۰ میلی‌ثانیه: موازی
const [user, posts] = await Promise.all([getUser(), getPosts()]);

// ✅ بهینه‌تر: همه را می‌خواهیم حتی اگر یکی شکست خورد
const results = await Promise.allSettled([getUser(), getPosts()]);
// [{status:"Fulfilled",value:...}, {status:"rejected",reason:...}]

// ✅ timeout با race:
const result = await Promise.race([fetch(url), wait(3000).then(() => { throw new Error("timeout") })]);

```

ابزار	رفتار
Promise.all	یکی شکست = همه reject
Promise.allSettled	همه را گزارش می‌دهد
Promise.race	اولین settle برنده است
Promise.any	اولین موفق برنده است

```
// ساده GET:
const res = await fetch("https://api.site.com/products");
const data = await res.json(); // ⚠️ دو مرحله: پاسخ → بدنه JSON

// POST با بدنه و هدر:
await fetch("https://api.site.com/orders", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify({ productId: 1, qty: 2 }),
});

// چک پاسخ - دو نکته‌ای که همه یادشان می‌رود:
// روی 404/500 ارور نمی‌دهد! خودت چک کن fetch ۱)
if (!res.ok) throw new Error(`HTTP ${res.status}`);
// لازم دارد await - است Promise هم خودش res.json() ۲)
```

۷.۵ – Event Loop: چرا این ترتیب؟ (سوال مصاحبه) ★

```
console.log("1 - sync");
setTimeout(() => console.log("4 - macrotask"), 0);
Promise.resolve().then(() => console.log("3 - microtask"));
console.log("2 - sync");
// خروجی: 1 → 2 → 3 → 4
```

ترتیب حلقه: **sync** → همه **microtask** ها (Promise ها) → یک **microtask** → **macrotask** (setTimeout) ها → ...

یعنی `setTimeout(fn, 0)` یعنی «در صف بعدی» نه «الان». و به همین دلیل `await` ها صفحه را فریز نمی‌کنند – فقط جریان آن تابع منطقاً منتظر می‌ماند.

جمع‌بندی فصل ✓

- چرا `async: single-thread` بودن؛ عملیات طولانی نباید بلاک کند
- `then/catch/finally`؛ `Promise: pending→fulfilled/rejected`
- `async/await`: خوانا؛ `try/catch`؛ `Promise.all` برای موازی (ضد waterfall)
- `fetch`: دو مرحله (پاسخ → json)، `res.ok` را چک کن، POST با `headers/body`
- event loop: `sync` → `microtask` → `macrotask`

تمرین فصل ۷ (با API رایگان <https://jsonplaceholder.typicode.com>) 📝

- `getUser(id)` : با `fetch` کاربر را بگیر و برگردان – با چک `res.ok` و `try/catch`.
- دو `fetch` موازی (`posts` و `users`) با `Promise.all` و خروجی ترکیبی.

3. تابع `withTimeout(promise, ms)` : اگر promise تا X ms جواب نداد، reject با «timeout» – با `Promise.race`.

4. خروجی را حدس بزن (فصل event loop):

```
(async () => {
  console.log("A");
  await Promise.resolve();
  console.log("B");
})();
console.log("C");
```

5. چالش واقعی: تابع `retry(fn, times)` بنویس – fetch را تا n بار تلاش کند با فاصله فزاینده (backoff).

▼ جوابها

```
// ۱
async function getUser(id) {
  try {
    const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("خطا:", err.message);
    return null;
  }
}

// ۲
const [users, posts] = await Promise.all([
  fetch("../users").then(r => r.json()),
  fetch("../posts").then(r => r.json()),
]);

// ۳
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("timeout")), ms)
  );
  return Promise.race([promise, timeout]);
}

// ۴: A C B – بعد از await، بقیه تابع به microtask می‌رود؛ C (sync) می‌شود.
// ۵
async function retry(fn, times = 3, delay = 500) {
  try {
    return await fn();
  } catch (err) {
    if (times <= 1) throw err;
    await wait(delay);
    return retry(fn, times - 1, delay * 2); // backoff فزاینده
  }
}
```

→ فصل بعد: ماژول‌ها، مدیریت خطای حرفه‌ای و دیباگ – پایان بخش JS!

فصل ۸ – ماژول‌ها، مدیریت خطا و دیباگ (پایان بخش JS)

🎯 هدف: کد را از یک فایل به پروژه‌ی واقعی برسانیم: ماژول‌های (import/export) ESM، استراتژی مدیریت خطا، و ابزارهای دیباگ حرفه‌ای. آخر این فصل، آماده‌ی ورود به TypeScript هستی.

۸.۱ – ماژول‌ها (ESM): تقسیم کد به فایل‌ها

هر فایل = یک ماژول با scope خودش. `export` = عرضه، `import` = مصرف:

```
// — math.js —
export function add(a, b) { return a + b; }           // نام‌دار export
export const PI = 3.14159;

export default function calculate(x, y) {             // پیش‌فرض - یکی در هر فایل export
  return { sum: add(x, y), area: PI * x * y };
}

// — app.js —
import calculate, { add, PI } from "./math.js";       // {} بدون {}, نام‌دار با default
import { add as sumFn } from "./math.js";            // تغییر نام
import * as math from "./math.js";                  // namespace همه به صورت

add(2, 3);
math.PI;
```

- مسیر نسبی: `./` هم‌پوشه، `../` والد – و پسوند `.js` را بنویس (در Node ESM)
- HTML: `<script type="module" src="app.js"></script>`
- یک `default` در هر فایل (چیز اصلی) + هر تعداد `export` نام‌دار

💡 در Next.js همین است – فقط مسیرها با `@/lib/...` (alias) خواناتر می‌شوند و `import` ها در `build` مدیریت می‌شوند.

```
// throw - پرتاب خطای سفارشی:
function divide(a, b) {
  if (b === 0) throw new Error("تقسیم بر صفر ممنوع");
  return a / b;
}

// try/catch/finally:
try {
  const r = divide(10, 0);
} catch (err) {
  console.error("خطا:", err.message);
} finally {
  console.log("همیشه اجرا می‌شود"); // پاکسازی
}

// خطاهای سفارشی با کلاس (برای تفکیک نوع خطا):
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}
```

استراتژی سه‌لایه:

لایه	کار
جلوگیری	validation ورودی (فصل‌های بعدی: zod در TS)
گرفتن محل	try/catch دقیق در مرزها (fetch، فرم)
گزارش	پیام تمیز به کاربر + لاگ کامل برای تو

⚠ هرگز `catch (e) {}` خالی – خطا را می‌بلعی و باگ نامرئی می‌شود. حداقل لاگ کن؛ بهتر: دوباره `throw` یا پیام معنادار.

```
// خانواده کامل console:
console.log("عادی");
console.warn("هشدار");
console.error("خطا");
console.table(users); // آرایه آبجکت‌ها به شکل جدول ★
console.group("جزئیات"); console.log("..."); console.groupEnd();
console.time("حلقه"); ...; console.timeEnd("حلقه"); // زمان‌گیری

// debugger - نقطه توقف در DevTools:
function tricky(data) {
  debugger; // باز است DevTools فقط وقتی اجرا اینجا متوقف می‌شود
  return data.map(process);
}

// خطاها را جدی بگیر: استک‌تریس را از بالا به پایین بخوان
// اولین فایل خود تو = محل شروع تحقیق
```

در DevTools (F12): تب breakpoint → Sources → با کلیک روی شماره خط → Watch متغیرها → Step (F10/F11). با `node --inspect app.js` همین ابزار در VS Code.

✓ جمع‌بندی فصل – پایان بخش JavaScript!

- ESM: `export/default` + `import` – هر فایل یک ماژول
- خطا: `throw` سفارشی + `try/catch/finally` + کلاس خطای اختصاصی + هرگز `catch` خالی
- دیباگ: Sources DevTools/time/debugger/console.table + خواندن `stack trace`
- تو حالا JS کامل بلدی: متغیر تا DOM، `async` تا ماژول – TS فقط لایه‌ی امنیتی روی همین است!

📝 تمرین فصل ۸

1. پروژه‌ات را سه فایلی کن: `math.js` (توابع فصل ۲)، `format.js` (فصل ۵) و `app.js` که `import` کند و استفاده کند.
2. کلاس `HttpError extends Error` با فیلد `status` بنویس؛ تابع `fetch` بنویس که روی پاسخ‌های بد آن را `throw` کند و مصرف‌کننده بر اساس `instanceof HttpError` پیام متفاوت بدهد.
3. در DevTools مرورگر: `console.table` را روی آرایه‌ای از ۳ آبجکت امتحان کن و یک breakpoint واقعی بگذار و متغیر را Watch کن.
4. باگ‌گیری: این کد چرا همیشه «خطا» می‌دهد؟

```
try {
  const data = await fetch(url).then(r => r.json());
  console.log(data);
} catch { console.log("خطا") }
```

(راهنما: `await` داخل `try` بدون تابع `async`؟)

▼ جواب تمرین ۴

`await` فقط در تابع `async` مجاز است – این کد `SyntaxError` می‌دهد (نه «خطا» را چاپ می‌کند). درست: تابع را `async` کن:

```
async function load(url) {
  try {
    const data = await fetch(url).then(r => r.json());
    console.log(data);
  } catch (err) {
    console.log("خطا:", err.message);
  }
}
```

🎓 **JavaScript تمام شد – وارد دنیای TypeScript می‌شویم:** همان JS، ولی با سپر تایپ!

فصل ۹ – TypeScript از صفر: نصب، annotate پایه

🎯 هدف بخش دوم: JS + TS = تایپ. همه‌ی JS فصل ۱-۸ را داری؛ حالا یاد می‌گیری *قبل از اجرا* خطاها را بگیر. نصب، annotate پایه، انواع، توابع – هر مثال JS قبلی، نسخه TS اش را می‌گیرد.

۹.۱ – چرا TypeScript؟ (با یک مثال دردناک)

```
// — JavaScript —
function getTotal(items) {
  return items.reduce((sum, i) => sum + i.price, 0);
}
getTotal("سلام"); // * صفحه کاربر - RUNTIME ارور در
getTotal([{ price: "ده" }]); // "ده" - باگ خاموش و مخرب!
```

کد بالا تا اجرا ساکت است – و بعد از deploy می‌میرد. TS همین را *قبل از اجرا* می‌گیرد:

```
// — TypeScript —
type Item = { price: number };

function getTotal(items: Item[]): number {
  return items.reduce((sum, i) => sum + i.price, 0);
}

getTotal("سلام"); // ✗ قبل از حتی ذخیره - EDITOR ارور در
getTotal([{ price: "ده" }]); // ✗ نمی‌رود number به string
getTotal([{ price: 50 }]); // ✓
```

TS هیچ‌چیز به runtime اضافه نمی‌کند – فقط **چک‌کننده‌ی زمان توسعه** است: کد TS کامپایل می‌شود به JS خالص.

🔑 جمله کلیدی: «TS فقط در زمان توسعه وجود دارد؛ خروجی نهایی JS خالص است.»

۹.۲ – نصب و اجرا

```
mkdir ts-playground && cd ts-playground
npm init -y
npm install -D typescript # نصب محلی (پیشنهادی)
npx tsc --init # ساخت tsconfig.json

npx tsc app.ts # می‌سازد: app.js کامپایل
npx tsc --watch # مانیتور: با هر ذخیره دوباره کامپایل
npx tsx app.ts # بدون کامپایل - برای تمرین/اسکرپت TS اجرای مستقیم
```

```
// متغیر:
let name: string = "علی";
let age: number = 25;
let isAdmin: boolean = false;
let ids: number[] = [1, 2, 3];
let pair: [string, number] = ["25", "علی"]; // tuple - نوع ثابت - طول و نوع ثابت

// دارد - اغلب لازم نیست بنویسی «type inference» TS: نکته مهم
let city = "تهران"; // TS خودش فهمید string
city = 5; // ❌ حذر! string ارور! چون از اول ❌
// نمی‌داند (پارامتر توابع!) inference قاعده: نوع پیش‌فرض را بنویس وقتی
```

نوع‌های ویژه

```
let nothing: undefined = undefined;
let missing: null = null;

// union - یکی از چند نوع:
let id: string | number;
id = "abc"; id = 42; // ✅ هر دو

// literal type - فقط این مقادیر:
let direction: "up" | "down" | "left" | "right";
direction = "up"; // ✅
direction = "diagonal"; // ❌

// any - دشمن TS (خاموش کردن)
let loose: any = "هرچی";
loose.doAnything(); // ❌ runtime چک نمی‌شود - باگ به

// unknown - narrow بگیر، ولی قبل مصرف any: نسخه امن
let safe: unknown = getData();
// safe.toUpperCase(); // ❌ اول بگو چه هستی!
if (typeof safe === "string") safe.toUpperCase(); // ✅

// void و never:
function log(msg: string): void { console.log(msg); } // هیچ برنمی‌گرداند
function fail(): never { throw new Error(); } // هرگز برنمی‌گردد
```

🔑 جمله مصاحبه‌ای: «TS any را خاموش می‌کند، unknown آن را روشن نگه می‌دارد - رازها را any نگیر،

unknown بگیر و narrow کن.»

```
// inference - پارامترها اجباری، خروجی معمولاً
function greet(name: string, excited: boolean = false): string {
  return excited ? `سلام ${name}!` : `سلام ${name}`;
}

// callback style: نوع کل تابع
type Mapper = (n: number) => string;
const map: Mapper = (n) => String(n); // پارامترها خودکار تایپ‌دار!

// annotation - مستقیم:
const user: { name: string; age: number; city?: string } = {
  name: "علی",
  age: 25, // city? اختیاری است - نداشتنش ارور نیست
};

// readonly - تغییر ممنوع:
const config: { readonly port: number } = { port: 3000 };
// config.port = 4000; // ❌
```

۹.۵ – tsconfig: قوانین پروژه

```
// tsconfig.json - حداقل‌های یک پروژه مدرن:
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true, // ★★ قوانین سخت - همیشه روشن!
    "noUncheckedIndexedAccess": true, // undefined آرایه‌ها ممکن است
    "skipLibCheck": true
  }
}
```

strict: true همیشه - پروژه‌های مدرن (Next.js) پیش‌فرض روشنش می‌کنند. strict خاموش = نصف TS را بیکار کرده‌ای.

جمع‌بندی فصل ✓

- TS = چک زمان توسعه؛ خروجی JS خالص؛ ارور قبل از اجرا
- نصب: `npm i -D typescript + tsconfig + npx tsc --watch` یا `tsx`
- annotation: متغیر/آرایه/never/void/unknown/literal/union/tuple
- inference: اغلب خودش می‌فهمد؛ پارامترها را تو بگو
- **strict: true** همیشه

تمرین فصل ۹

1. playground بساز و سه نسخه بنویس: نسخه JS باگ‌دار (`getTotal`)، بعد همان با TS – و ارورهای editor را ببین.
2. برای این تابع TS بنویس: `function repeat(text, times)` – تایپ کامل با پیش‌فرض.
3. union تمرین: متغیر `status: "idle" | "loading" | "success" | "error"` – مقادیر معتبر/نامعتبر را تست کن و ارورها را بخوان.
4. فرق `any` و `unknown` را عملاً نشان بده: روی هر دو `.toUpperCase()` صدا بزن – کدام ارور می‌دهد؟ چرا این یعنی امن‌تر؟
5. تایپ این آبجکت را کامل کن (شهر اختیاری، تگ‌ها آرایه رشته):
`const user = { name: "علی", age: 25, city: "تهران", tags: ["js"] }`

جواب‌ها ▼

```
// ۲
function repeat(text: string, times: number = 1): string {
    return text.repeat(times);
}

// ۴: any بدون ارور اجرا/کامپایل می‌شود و اگر رشته نباشد در any
//      «Object is of type 'unknown'» - بلافاصله ارور می‌دهد unknown
//      گرفته می‌شود editor یعنی باگ در (typeof) مجبور می‌شوی اول چک کنی

// ۵
const user: {
    name: string;
    age: number;
    city?: string;
    tags: string[];
} = { name: "علی", age: 25, city: "تهران", tags: ["js"] };
```

→ فصل بعد: union، interface/type‌های پیشرفته، narrowing و generics.

فصل ۱۰ – Generics و interface/type، Union

🎯 هدف: ابزارهای اصلی مدل‌سازی داده در TS: تعریف شکل‌ها با interface/type، ترکیب با union/intersect، narrowing امن، و generics. بعد از این فصل می‌توانی هر داده‌ای را تایپ کنی.

۱۰.۱ – interface در برابر type

```
// برای شکل آبجکت‌ها - interface:
interface User {
  id: number;
  name: string;
  email?: string; // JS اختیاری - مثل ? در
  readonly createdAt: Date; // فقط خواندنی
}

// type - انعطاف بیشتر (union, primitive, utility):
type ID = string | number;
type Status = "active" | "banned";
type Users = Record<string, User>; // utility (فصل ۱۱)

// هر دو قابل استفاده‌اند:
const u: User = { id: 1, name: "علی", createdAt: new Date() };
const uid: ID = 42;
```

type	interface	
✓	✓	آبجکت
✓	✗	union/literal
&	extends	extends / intersection
✗ (ارور تکرار)	✓ (دوباره تعریف = ادغام)	declaration merging

💡 قاعده تیمی: برای شکل آبجکت‌ها interface (قابل extends و merge)، برای بقیه type. مهم‌تر: یکدستی در پروژه.

```
interface Person { name: string }
interface Employee { salary: number }

interface Staff extends Person, Employee {}           // interface ارث‌بری
type StaffT = Person & Employee;                     // intersection type - همان معنا

const s: Staff = { name: "علی", salary: 500 };
```

۱۰.۲ – Narrowing و Union: نوع شرطی امن ★

```
type Result = { ok: true; data: string[] } | { ok: false; error: string };

function show(res: Result) {
  // res.data ❌ - نمی‌دانی کدام شکل است!
  if (res.ok) {
    console.log(res.data);           // ✅ TS فهمید شکل اول است
  } else {
    console.error(res.error);        // ✅ و اینجا شکل دوم
  }
}
```

یک **narrowing** است - TS بر اساس چک، نوع را تنگ‌تر می‌کند. انواع narrowing: `if (res.ok)`

```
function process(input: string | number | null) {
  if (typeof input === "string") input.toUpperCase(); // typeof
  if (input === null) input.valueOf();                // چک null
  if (Array.isArray(input)) {}                       // آرایه
  if (input instanceof Date) input.getTime();         // کلاس
}
```

Discriminated union – الگوی طلایی (این را حفظ کن!):

```
type FetchState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; message: string };

function render(state: FetchState<string[]>) {
  switch (state.status) { // کلید تمایز
    case "idle": return "شروع نشده";
    case "loading": return "در حال بارگذاری";
    case "success": return state.data.join(","); // ✅ فقط اینجا موجود data
    case "error": return state.message;
  }
}
```

چرا طلایی؟ چون TS فراموش کردن یک حالت را می‌گیرد و در هر شاخه فقط فیلدهای همان حالت را می‌دهد – دقیقاً ساختار state واقعی اپ‌ها.

```
// enum - ثابت نامدار - لیست:
enum OrderStatus {
  Pending = "PENDING",
  Paid = "PAID",
}
let s: OrderStatus = OrderStatus.Paid;

// ★ مدرن تر و سبک تر - union + as const:
const ORDER_STATUS = {
  pending: "PENDING",
  paid: "PAID",
} as const;
// فقط خواندنی می شوند literal مقادیر
type OrderStatus2 = (typeof ORDER_STATUS)[keyof typeof ORDER_STATUS];
// "PENDING" | "PAID"
```

(as const و typeof نوع سازی پیشرفته اند - فعلاً الگو را ببین؛ فصل ۱۱ عمیق می شود).

۱۰.۴ Generics: تایپ پارامتری ★

مشکل: `function first(arr)` - ورودی/خروجی چه تایپی؟ `any` می گیریم و type safety می میرد. جواب:

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

const n = first([1, 2, 3]); // n: number | undefined - خودش فهمید!
const s = first(["a"]);     // s: string | undefined
```

`<T>` = «پارامتر تایپ». TS از آرگومان حدس می زند (inference) - مثل پارامترهای عادی ولی برای نوع:

```
// generic + محدودیت با extends:
function pick<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
const user = { name: "علی", age: 25 };
pick(user, "name"); // ✅ type: string
pick(user, "email"); // ❌ "email" در keyof T نیست!

// generic در interface - جواب API:
interface ApiResponse<T> {
  data: T;
  status: number;
}
const res: ApiResponse<User> = await fetchUser();
res.data.name; // ✅ تایپ سیف کامل!
```

generic های بومی که هر روز می بینی: `useState<string>(...)`، `Map<K,V>`، `Promise<T>`، `Array<T>`،

`useActionState<TState, ...>` - همه همین الگو.

جمع‌بندی فصل ✓

- interface برای شکل آبجکت (+ extends)، type برای union و بقیه
- union + narrowing = امنیت واقعی؛ **discriminated union** با switch الگوی طلایی state
- enum مدرن = union literal + as const
- **generics** = تایپ پارامتری: `first<T>(arr: T[])`، `ApiResponse<T>`، `K extends keyof T`
- همه ابزارهای بومی (Array/Promise/useState) generics اند

تمرین فصل ۱۰ 📝

1. `Product` interface بساز: `id`، `title`، `price`، `category`، درست‌کردن یک محصول و یک ارور عمدی (فیلد غلط) – ارور editor را بخوان.
2. `discriminated union` برای «پرداخت»: حالت‌های `idle/loading/success` (شناسه تراکنش)/`error` (پیام) – تابع `render` با `switch` بنویس.
3. تابع `wrapInArray<T>(value: T): T[]` بنویس و با سه نوع مختلف تست کن.
4. `generic` محدود: `function pluck<T, K extends keyof T>(items: T[], key: K): T[K][]` – روی `products` با `"title"` کار کند و با کلید غلط ارور بدهد.
5. `User` interface را با `extends` به `AdminUser` گسترش بده (`permissions: string[]`).

```
// ۱
interface Product {
  id: number;
  title: string;
  price: number;
  category: string;
}

const p: Product = { id: 1, title: "گوشی", price: 50, cat: "mobile" };
// ❌ 'cat' does not exist in type 'Product'. Did you mean 'category'?

// ۲
type PaymentState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; txId: string }
  | { status: "error"; message: string };

function renderPay(state: PaymentState): string {
  switch (state.status) {
    case "idle": return "آماده پرداخت";
    case "loading": return "در حال پردازش ";
    case "success": return `موفق ✅ ${state.txId} تراکنش`;
    case "error": return `خطا: ${state.message}`;
  }
}

// ۳
function wrapInArray<T>(value: T): T[] {
  return [value];
}

wrapInArray(5); wrapInArray("a"); wrapInArray({ x: 1 });

// ۴
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}

pluck(products, "title"); // string[]
pluck(products, "email"); // ❌ ارور کامپایل!

// ۵
interface AdminUser extends User {
  permissions: string[];
}
```

فصل ۱۱ – TS پیشرفته: Utility Types، Type Guards و satisfies

🎯 هدف: ابزارهایی که کد TS را از «تایپ شده» به «حرفه‌ای» تبدیل می‌کنند: Utility Types آماده، type guard اختصاصی، satisfies و چند الگوی واقعی. این فصل = مرز متوسط تا پیشرفته.

۱۱.۱ – Utility Types: ترنسفورمرهای آماده ★

از یک تایپ، تایپ‌های جدید بساز – بدون تکرار:

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// Partial – همه فیلدها اختیاری (فرم آپدیت!):
function updateUser(id: number, changes: Partial<User>) {
  // { name?: string, email?: string, ... } – فقط فیلدهای فرستاده‌شده
}
updateUser(1, { name: "علی جدید" });

// Required – همه اجباری:
// Pick – فقط بعضی فیلدها:
type UserCard = Pick<User, "id" | "name">; // { id, name } – کارت محصول!

// Omit – حذف بعضی فیلدها:
type PublicUser = Omit<User, "email">; // بدون ایمیل (حریم!)

// Record – دیکشنری با شکل مشخص:
const roles: Record<string, string[]> = {
  admin: ["read", "write"],
  viewer: ["read"],
};

const counts: Record<"a" | "b", number> = { a: 1, b: 2 };

// Readonly – همه فقط‌خواندنی:
const frozen: Readonly<User> = { ... };
// frozen.name = "x" // ❌

// ReturnType / Parameters – تایپ از دل تابع:
function createOrder(input: { productId: number; qty: number }) {
  return { id: 1, ...input, total: input.qty * 100 };
}
type Order = ReturnType<typeof createOrder>; // { id, productId, qty, total }
type OrderInput = Parameters<typeof createOrder>[0]; // بدون تکرار تایپ ورودی!
```

🔑 فلسفه: Single source of truth تایپ‌ها — User را یک‌بار بنویس و با utility ها مشتق کن؛ تغییر آینده فقط یک‌جا.

۱۱.۲ — Type Guard اختصاصی: is

وقتی TS نمی‌تواند خودش تشخیص دهد (مثل داده‌ی API)، به آن دست بده:

```
interface Cat { meow(): void }
interface Dog { bark(): void }

function isCat(animal: Cat | Dog): animal is Cat { // ★ type predicate
  return "meow" in animal;
}

function speak(animal: Cat | Dog) {
  if (isCat(animal)) animal.meow(); // ✅ TS می‌داند Cat است
  else animal.bark();
}
```

کاربرد واقعی — پاسخ API:

```
interface ApiUser { id: number; name: string }

function isApiUser(data: unknown): data is ApiUser {
  return (
    typeof data === "object" && data !== null &&
    "id" in data && "name" in data
  );
}

const raw: unknown = await getData();
if (isApiUser(raw)) {
  console.log(raw.name); // ✅ امن و تایپ‌دار
}
```

(در پروژه‌های جدی، این کار را به zod می‌سپاری که هم validate هم تایپ تولید کند — فصل ۱۲).

۱۱.۳ — satisfies: بهترین هر دو دنیا (+TS 4.9)

مشکل کلاسیک: annotation نوع را محدود می‌کند (autocomplete از دست می‌رود)، بدون annotation هم چک نیست.

satisfies چک می‌کند بدون از دست دادن نوع دقیق:

```

type Routes = "/home" | "/about" | "/contact";

// ❌ با annotation: مقادیر دقیق گم می‌شوند
const routes1: Record<Routes, { title: string }> = {
  "/home": { title: "خانه" },
  "/about": { title: "درباره" },
  "/contact": { title: "تماس" },
};
// routes1["/home"].title → string (خوب) اما.

// ✅ با satisfies: چک + حفظ نوع دقیق
const routes2 = {
  "/home": { title: "خانه" },
  "/about": { title: "درباره" },
  "/contact": { title: "تماس" },
} satisfies Record<Routes, { title: string }>;

routes2["/home"].title; // کامل autocomplete - نوع دقیق حفظ شده
// و اگر کلیدی جا بیفتد یا اضافه شود → ارور کامپایل

```

۱۱.۴ – الگوهای پیشرفته‌ی پرکاربرد

```

// keyof – یک تایپ: کلیدهای یک تایپ:
type UserKeys = keyof User; // "id" | "name" | "email" | "age"

// lookup type – نوع یک فیلد:
type userEmail = User["email"]; // string

// تایپ آرایه از مقدار:
const colors = ["red", "green", "blue"] as const;
type Color = (typeof colors)[number]; // "red" | "green" | "blue"

// exclude از union:
type EditableKeys = Exclude<keyof User, "id">; // بدون id!

```

۱۱.۵ – strict mode و نکات دیباگ TS

```

// strict: true یعنی این‌ها فعال‌اند:
// strictNullChecks: null/undefined (مهم‌ترین!)
// noImplicitAny: (any ممنوع) نفهمید، ارور بده TS هر جا
// را چطور بخوانی TS ارورهای:
// "Type 'string' is not assignable to type 'number'"
// = رشته به عدد نمی‌رود - خطا را از انتها به ابتدا بخوان

// وقتی «تو بهتر می‌دانی» (کمتر استفاده کن!):
const el = document.getElementById("input") as HTMLInputElement;
el.value = "...";
// ⚠️ as یعنی «تو بهتر می‌دانی» - مسوولیت باگ با توست TS
// (!نیست) - با چک پشتش null یعنی «قطعاً x!» (non-null assertion)

```

جمع‌بندی فصل ✅

• Utility ها: Partial (آپدیت) / ReturnType / Readonly / Record / Omit / Pick

- **Single source of truth**: تایپ پایه + مشتقات
- type guard با `is` برای داده (API) unknown
- `satisfies` = چک بدون از دست دادن دقت
- `keyof / lookup / as const / Exclude` – ترکیب‌ها قدرت واقعی‌اند
- `strict: true` همیشه؛ `as` و `!` را کم و آگاهانه استفاده کن

تمرین فصل ۱۱

1. از `interface Invoice { id, customerId, total, createdAt }` : تایپ‌های `InvoiceDraft` (بدون `id` و `createdAt`)، `InvoiceSummary` (فقط `id` و `total`) و دیکشنری `InvoicesById` بساز – با `utility` ها.
2. type guard بنویس: `isStringArray(data: unknown): data is string[]` – و در تابع مصرفی استفاده‌اش کن.
3. `satisfies` را روی یک آبجکت `config` با کلیدهای `union` امتحان کن – یک کلید غلط اضافه کن و ارور را ببین.
4. با `as const` و `lookup`، تایپ `Theme = "light" | "dark"` را از یک آرایه استخراج کن (بدون نوشتن `union` دستی).
5. چالش: تابعی بنویس که فقط کلیدهای `string` دار یک آبجکت را بپذیرد: `K extends keyof T` + شرط `T[K]` `.extends string`

```
// ۱
type InvoiceDraft = Omit<Invoice, "id" | "createdAt">;
type InvoiceSummary = Pick<Invoice, "id" | "total">;
type InvoicesById = Record<string, Invoice>;

// ۲
function isStringArray(data: unknown): data is string[] {
    return Array.isArray(data) && data.every((x) => typeof x === "string");
}

// ۳
type Page = "home" | "about";
const titles = {
    home: "خانه",
    about: "درباره",
    extra: "X" // ارور: Object literal may only specify known keys
} satisfies Record<Page, string>;

// ۴
const themes = ["light", "dark"] as const;
type Theme = (typeof themes)[number]; // "light" | "dark"

// ۵
function pickStrings<T, K extends keyof T>(obj: T, key: K): T[K] | undefined {
    // راه تمیز با قید:
    function stringProps<T extends Record<string, unknown>, K extends keyof T>(
        obj: T, key: K
    ): T[K] {
        return obj[key];
    }
}
```

➔ فصل بعد: TS در پروژه واقعی — تایپ API، کامپوننت‌های React و zod!

فصل ۱۲ – TypeScript در پروژه واقعی: React، API و zod

🎯 هدف: جمع‌بندی عملی – سه سناریویی که هر روز در کار واقعی می‌بینی: تایپ‌کردن پاسخ API، تایپ‌کردن کامپوننت‌ها و هوک‌های React، و اعتبارسنجی داده ورودی با zod. این فصل = پل TS به پروژه‌های React/Next تو.

۱۲.۱ – تایپ‌کردن پاسخ API

مساله: `fetch` همیشه `any` می‌دهد. سه لایه حل:

```
// تایپ داده – مطابق قرارداد بک‌اند (۱)
interface Product {
  id: number;
  title: string;
  price: number;
  inStock: boolean;
  category: { id: number; name: string };
}

// پاسخ wrapper تایپ (۲)
interface ApiList<T> {
  data: T[];
  total: number;
  page: number;
}

// تایپ‌دار fetch تابع (generic) (۳):
async function fetchJson<T>(url: string): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json() as Promise<T>; // ⚠️ as: تو به قرارداد API اعتماد می‌کنی
}

// مصرف – تایپ‌سیف کامل:
const res = await fetchJson<ApiList<Product>>("/api/products");
res.data[0].title; // ✅ autocomplete کامل و چک
res.data[0].titel; // ❌ ارور کامپایل – غلط املایی گرفته شد
```

⚠️ نکته صادقانه: `as Promise<T>` یعنی «به API اعتماد می‌کنم که همین شکل را می‌فرستد». اگر API غیرقابل‌اعتماد است یا نسخه‌اش عوض می‌شود → zod (بخش ۱۲.۳) داده را در runtime هم چک می‌کند.

ابزار طلایی: [GraphQL Code Generator] یا [OpenAPI typescript] تایپ‌ها را از سرویس خود API تولید می‌کنند – صفر دست‌نویسی.

```
// props کامپوننت - با interface:
interface ProductCardProps {
  product: Product;
  onSelect?: (id: number) => void;    // callback اختیاری
  highlighted?: boolean;
}

function ProductCard({ product, onSelect, highlighted = false }: ProductCardProps) {
  return (
    <div
      className={highlighted ? "ring-2" : ""}
      onClick={() => onSelect?.(product.id)} // ? برای callback اختیاری
    >
      <h3>{product.title}</h3>
      <span>{product.price.toLocaleString("fa-IR")}</span>
    </div>
  );
}
```

```
// رویدادها - تایپ‌های React:
import type { ChangeEvent, FormEvent, KeyboardEvent } from "react";

function SearchBox() {
  const [query, setQuery] = useState("");

  function handleChange(e: ChangeEvent<HTMLInputElement>) {
    setQuery(e.target.value);    // ✅ autocomplete کامل
  }

  function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
  }

  function onKey(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === "Enter") submit();
  }

  // ...
}
```

```
// نمی‌داند inference تایپ‌دار - صریح وقتی useState:
const [user, setUser] = useState<User | null>(null);    // کاربر یا هیچ
const [status, setStatus] = useState<"idle" | "loading">("idle");
const [items, setItems] = useState<Product[]>([]);

// (Single source of truth): ترفند: تایپ را از داده استخراج کن
type Product = typeof products[0];    // اگر products از جایی import شده!

// با نوع useRef:
const inputRef = useRef<HTMLInputElement>(null);
```

برای `React.ReactNode` و هر JSX ای؛ `React.ComponentProps<typeof Button>` برای «همه
props های یک کامپوننت دیگر» - ترکیب به جای تکرار! 💡

۱۲.۳ – zod: اعتبارسنجی runtime + تایپ خودکار ✨ ✨

مشکل باقی‌مانده: تایپ‌های TS در runtime وجود ندارند – داده‌ی API/فرم کاربر را در زمان اجرا باید چک کنی. zod هر دو را یک‌جا می‌دهد:

```
npm i zod
```

```
import { z } from "zod";

// ۱. اسکیمای را تعریف کن:
const ProductSchema = z.object({
  id: z.number().int().positive(),
  title: z.string().min(1),
  price: z.number().nonnegative(),
  tags: z.array(z.string()).default([]),
});

// ۲. تایپ را از اسکیمای استخراج کن – نوشتن دستی ممنوع:
type Product = z.infer<typeof ProductSchema>;
// { id: number; title: string; price: number; tags: string[] } – دقیق و همیشه همگام!

// ۳. اعتبارسنجی کن در runtime:
const result = ProductSchema.safeParse(apiResponse);
if (result.success) {
  console.log(result.data.title); // validated تایپ‌دار + واقعاً ✅
} else {
  console.error(result.error.issues[0].message); // "Invalid input..."
}

// فصل ۸ مرور Server Action همان الگوی zod فرم با:
const FormSchema = z.object({
  title: z.string().min(3, "عنوان حداقل ۳ حرف"),
  price: z.coerce.number().positive("قیمت باید مثبت باشد"), // coerce: "۱۲" → ۱۲
});

const parsed = FormSchema.safeParse(Object.fromEntries(formData));
```

جریان حرفه‌ای کامل: zod اسکیمای `z.infer` → تایپ → validation در مرز (فرم/API) → بعد از آن همه‌جا تایپ‌سیف مطمئن. (در Next: همین اسکیمای در Server Action مصرف می‌شود).

۱۲.۴ – چک‌لیست TS در پروژه واقعی

- `strict: true` – همیشه
- [] تایپ‌های دامنه در یک فایل (`types/`) – از API/اسکیمای zod مشتق، نه دستی دوباره‌نویسی
- [] `any` ممنوع؛ `narrowing` + `unknown` یا zod
- [] `as` فقط با دلیل مستندشده
- [] utility ها به‌جای تکرار تایپ
- [] `z.infer` = پل runtime و compile time

✓ جمع‌بندی فصل – و پایان بخش TS!

- fetch تایپ‌دار با `fetchJson<T>` generic: + تایپ پاسخ – و zod برای اعتماد runtime
- React: interface props، event types، `typeof` برای استخراج
- DOM با `useRef`؛ union literal؛ `useState` با نوع
- zod: schema → `z.infer` → `safeParse` – تایپ و validation از یک منبع
- TS واقعی یعنی: داده از مرز (API/کاربر) وارد می‌شود → `validate` می‌شود → بعدش همه‌چیز تایپ‌سیف

📝 تمرین فصل ۱۲

1. برای `https://jsonplaceholder.typicode.com/todos` API تایپ `Todo` بساز (`id, userId, title, completed`) و `fetchJson<Todo[]>` را بنویس و مصرف کن – `autocomplete` را ببین.
2. کامپوننت `TodoItem` با props تایپ‌دار (`todo + onToggle`) بنویس – و یک props غلط بده تا ارور کامپایل را ببینی.
3. zod schema برای فرم تماس (نام، ایمیل با `.email()`، پیام `min 10`) بنویس؛ `z.infer` بگیر و یک داده غلط را `safeParse` کن – پیام‌های ارور را بخوان.
4. چالش: `fetchJson` را با zod ترکیب کن: `fetchValidated<T>(url, schema: z.ZodType<T>): Promise<T>` –
که هم `fetch` کند هم `validate`.

```
// ۱
interface Todo {
  id: number;
  userId: number;
  title: string;
  completed: boolean;
}
const todos = await fetchJson<Todo[]>("https://jsonplaceholder.typicode.com/todos");
todos.filter((t) => !t.completed).length; // ✅ همه چیز autocomplete

// ۲
interface TodoItemProps {
  todo: Todo;
  onToggle: (id: number) => void;
}
function TodoItem({ todo, onToggle }: TodoItemProps) {
  return (
    <li onClick={() => onToggle(todo.id)}>
      {todo.completed ? "✅" : "❌"} {todo.title}
    </li>
  );
}
// <TodoItem todo={5} .../> → ❌ Type 'number' is not assignable to type 'Todo'

// ۳
const ContactSchema = z.object({
  name: z.string().min(1, "نام الزامی است"),
  email: z.string().email("ایمیل نامعتبر"),
  message: z.string().min(10, "پیام حداقل ۱۰ کاراکتر"),
});
type Contact = z.infer<typeof ContactSchema>;
ContactSchema.safeParse({ name: "", email: "bad", message: "hi" });
// success: false - issues: سه پیام فارسی بالا

// ۴
async function fetchValidated<T>(url: string, schema: z.ZodType<T>): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const raw: unknown = await res.json();
  return schema.parse(raw); // parse یا throw - با پیام دقیق
}
```

🎓 TS تمام شد! فصل آخر: چیت‌شیت دوتایی و نقشه راه.

فصل ۱۳ – چیت‌شیت دوتایی، گلاساری و نقشه راه

🎯 هدف: مرجع مرور سریع دو زبان – JS در یک صفحه، TS در یک صفحه – به‌علاوه گلاساری و نقشه راه.

```
// == متغیرها ==
const x = 1;           // پیش فرض
let y = 2;             // قابل تغییر
// var هرگز

// == عملگرها ==
===               // همیشه
??               // nullish (نه || )
% **             // باقیمانده / توان
"5" + 3 // "53" ⚠️

// == شرط و حلقه ==
if (...) {} else if {} else {}
cond ? a : b
for (let i = 0; i < n; i++)
for (const item of items)
while (cond)
break / continue

// == توابع ==
function f(a, b = 0) { return a + b }
const f = (a) => a * 2;           // return ضمنی
function f(...args) {}          // rest

// == closure ==
function make() { let n = 0; return () => ++n; }

// == آبجکت/آرایه ==
const { a, b = 0 } = obj;        // destructuring
const [x, ...rest] = arr;
const copy = { ...obj, k: v };   // shallow!
arr.at(-1).push().includes()
Object.keys(obj) / entries(obj)

// == آرایه متدها ==
.map(fn)           // تبدیل
.filter(fn)        // حذف
.find(fn)          // اولین
.reduce((acc, x) => ..., init) // تاخوردن
.some(fn) / every(fn)
sort((a,b) => a - b)
groupBy: reduce + (acc[k] ??= []).push(x)

// == رشته ==
s.trim().split(",").join(" ").replaceAll("a","b").includes().at(-1)
`${x + 1}`          // template literal

// == اعداد Intl ==
Number("42").parseInt parseFloat
Number.isNaN(x)
new Intl.NumberFormat("fa-IR").format(n)

// == Async ==
async function f() {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(res.status);
    return await res.json();
  } catch (e) { ... } finally { ... }
}
await Promise.all([a, b])        // موازی!
// event loop: sync → microtask → macrotask

// == ماژول ==
export default fn; export const x = 1;
```

```
import fn, { x } from "./mod.js";

// == DOM ==
document.querySelector("#id")
el.textContent = "..." // امن
el.classList.toggle("c")
el.addEventListener("click", fn)
e.preventDefault() // در submit
```

۱۳.۲ – چیت‌شیت TypeScript

```
// == پایه ==
let s: string = "x";
let ids: number[] = [1, 2];
let pair: [string, number] = ["a", 1];
let id: string | number; // union
let dir: "up" | "down"; // literal
function f(a: string, b = 0): string {...}
// any × / unknown + narrowing ✓

// == interface / type ==
interface User { id: number; email?: string; readonly createdAt: Date }
type ID = string | number;
type Staff = Person & Employee; // intersection
interface Staff extends Person {}

// == narrowing ==
typeof x === "string" / Array.isArray / instanceof
if (res.ok) { res.data } // discriminated union + switch
function isCat(a: Cat | Dog): a is Cat { return "meow" in a }

// == generics ==
function first<T>(arr: T[]): T | undefined
interface ApiResponse<T> { data: T }
function pluck<T, K extends keyof T>(o: T, k: K): T[K]

// == utility types ==
Partial<T> // همه اختیاری (آپدیت)
Pick<T, "a" | "b"> // فقط بعضی
Omit<T, "id"> // حذف بعضی
Record<string, T> // دیکشنری
readonly<T> / Required<T>
ReturnType<typeof fn> / Parameters<typeof fn>[0]

// == ترفندها ==
const colors = ["red", "green"] as const;
type Color = (typeof colors)[number];
type UserEmail = User["email"];
{ ... } satisfies Record<K, V> // چک + حفظ دقت
as HTMLElement // کم و آگاهانه - assertion

// == zod ==
const Schema = z.object({ name: z.string().min(1) });
type Data = z.infer<typeof Schema>; // تایپ از اسکیمایما
Schema.safeParse(input); // runtime validation
```

انگلیسی	فارسی	یک جمله
Variable	متغیر	جعبه‌ای با اسم برای مقدار
Scope	دامنه دید	متغیر کجا زندگی می‌کند
Hoisting	بالا آمدن	تعریف‌ها قبل اجرا بالا می‌آیند
Closure	بستار	تابع + حافظه‌ی محیط تولدش
Callback	تابع برگشت‌خوانی	تابعی که به تابع دیگر داده می‌شود
Promise	وعده	رسید نتیجه‌ی عملیات آینده
async/await	—	نوشتن async به شکل خوانا
Event Loop	حلقه رویداد	چرخه اجرای sync/microtask/macrotask
Destructuring	بازکردن ساختار	استخراج تمیز فیلد/عنصر
Spread/Rest	پخش/جمع ...	باز کردن / جمع کردن
Shallow copy	کپی سطحی	سطح اول کپی؛ تو در تو مرجع مشترک
Immutable	تغییرناپذیر	به‌جای تغییر، نسخه جدید
Type Inference	استنتاج نوع	TS خودش نوع را حدس می‌زند
Union	اجتماع نوع	<code>string number</code>
Narrowing	تنگ کردن نوع	با چک، نوع دقیق‌تر می‌شود
Generic	نوع عمومی/پارامتری	<code>function first<T></code>
Type Guard	نگهبان نوع	تابعی که نوع را تضمین می‌کند (<code>is</code>)
Utility Type	تایپ کمکی	<code>...Partial/Pick/Omit</code>
strict mode	حالت سخت‌گیر	قوانین کامل TS روشن
Zod	—	اعتبارسنجی runtime + تایپ با infer



سه توصیه پایانی:

1. TS را با سخت‌گیری یاد بگیر، نه با فرار: `any` و `as` وسوسه‌انگیزند – هر بار که استفاده کردی بپرس «راه درستش چی بود؟» – همین تمرین، تو را senior می‌کند.
 2. JS عمیق = TS سریع: closure، event loop و متدهای آرایه را که واقعاً فهمیده باشی، TS برایت فقط قوانین است، نه درس جدید.
 3. داکيومنت رسمی دوست توست: developer.mozilla.org (MDN) برای JS (بهترین مرجع دنیا – حتی فارسی‌سازی دارد) و typescriptlang.org/docs – هر دویی که این دوره کوتاهش کرده.
- پایان دوره:** تو از «متغیر یعنی چی؟» به «generic با constraint و utility type» رسیدی – و حالا در هر پروژه‌ای، هر زبانی از این دو، راحت کد می‌زنی. قدم بعدی در کتابخانه‌ات: دوره ۶ (React/Next) را با چشم TS دوباره بخوان – همه‌چیز عمیق‌تر دیده می‌شود! 🚀 ♦️ 🟡